First Steps

1. Open the Playground scene in the Scenes folder
2. Play it!
3. You can mess around with the variables, especially in the Player and Cameras GameObject hierarchies
4. It is recommended that you turn the Playground scene into your real project, as trying to place the player and camera prefabs in an existing project would more often than not create a confusing mess

Modifying Inputs

1. Assets > Burntwax Collective > Burntwax FPS Demo > Scripts > Player > Input > **Manager**
2. 

3. Go to this path and double click the PlayerInput, which will pop up a tab with all the input settings. In there, you can change all the inputs to whatever you'd like
4. **If you need to create new inputs** for new mechanics, you must go to the InputManager script, and then create methods for subscribing/unsubscribing to actions
    a. You can follow by the examples of all the other inputs, so it doesn't require any real coding

Player Movement

1. We use a hierarchical state machine, where there are "root" super states and then sub states. The player is in both a root state and a sub state at all times.
2. PlayerGroundedState, PlayerJumpState, PlayerFallState, PlayerSlopeState, Player WallRunState, etc. are the root states. The sub states are states like PlayerWalkState and PlayerIdleState which can happen during any of the root states. When you are grounded, you can be walking, sprinting, sitting idle, or crouching. But you can also do this while wall running, falling, or jumping, and add any exceptions you require (ex: no crouching while jumping/falling)
    a. All states, super or sub, are added to the PlayerStateFactory script. The difference between them happens in the individual state script.

b. Root state:

```
Tabnine | Edit | Test | Explain | Document | 1 reference
public PlayerGroundedState(PlayerStateMachine currentContext, PlayerStateFactory
: base(currentContext, playerStateFactory)
{
    IsRootState = true;
}
```

c. Sub state:

```
Tabnine | Edit | Test | Explain | Document | 1 reference
public PlayerSprintState(PlayerStateMachine currentContext, PlayerStateFactory
: base(currentContext, playerStateFactory)
{}
```

d. That's it! Otherwise all the logic is the same.

3. **If you need to add a new state**, then go to PlayerStateFactory and add in the state in the enums, then follow the examples to create their contexts and functions. Then create your new c# file based off of any of the example states, and in the functions such as EnterState(), UpdateState(), and CheckSwitchStates(), you can add all the necessary logic for switching between states, coding any new mechanics, etc.
4. PlayerStateMachine is where all the general code lives, as some functions require constant updating regardless of state. Ground and slope checking functions are some examples, but you can put whatever function that needs constant updating in there.

Cameras

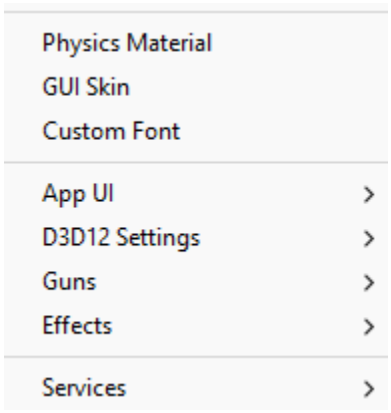
1. We use cinemachine vcams for our FPS character
    a. FPS cam for normal settings, an aim cam with decreased FOV for aiming, and you can add as many other cameras as you want for your uses
2. Cameras rely on the same hierarchical state machine structure as player movement, so it's easy to connect player movement states to camera states
3. **To use a new camera** with unique settings, just create a new vcam and add the settings you require. Then attach the new camera to the CamStateMachine script in the Player GameObject. Just like in our Player Movement section add the new camera state in the StateFactory and follow the other examples to set up context and functions. After that you can create your c# file and add all the necessary logic for your new state.

Guns

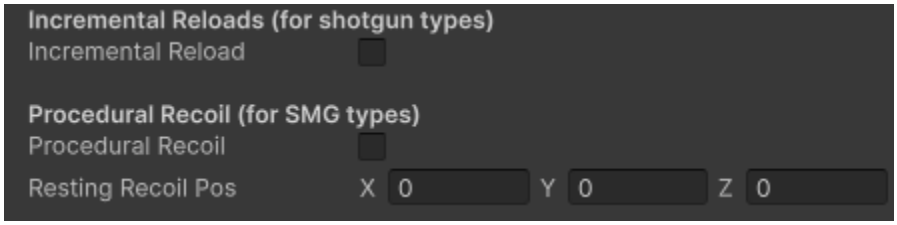
1. Unsurprisingly, we use the same hierarchical structure for the individual gun states in our system
2. For the system as a whole, we use Scriptable Objects to create new guns and their configurations for easy modification and creation

3. **To create a new gun**, go into this path here

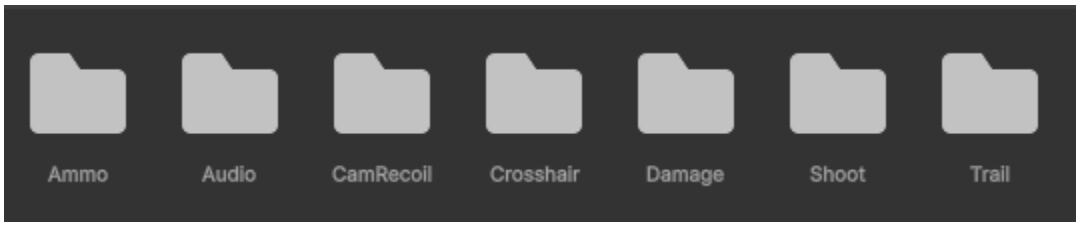


4. 

If you right-click → create you will find you can create a gun Scriptable Object.
5. Once you name it, you should set the ID to whatever the next incremental number is. It is important that you set it to whatever you want the actual loadout position to be, ex: if you want the Pistol gun first then set the ID to 0, if you want the Shotgun 4th then set it to 3.
6. For naming purposes, go into the GunType file and add in a new enum name which you can then set in the Scriptable Object.
7. There are also some specialized mechanics for certain guns. For example, shotguns can reload their ammo one shell at a time, and players can interrupt this by shooting. And SMGs usually have random recoil patterns. While our standard guns use standard Unity animations for recoils, if you check the box then you can use custom procedural recoil for your gun.



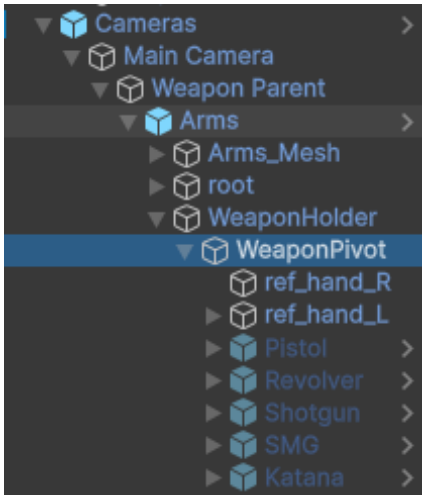
8. For the configs, go to the create → Guns and then create each config. You can find the example configs in these folders:



9. It's pretty self-explanatory, just configure each Scriptable Object how you want and then it'll apply to your gun. Just remember to set the right config to the right gun here:

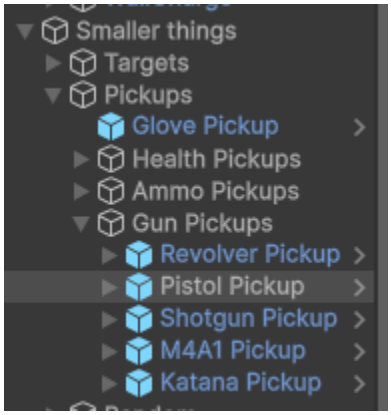


10. Once you've finished setting up your Scriptable Object, go to the scene hierarchy path here:



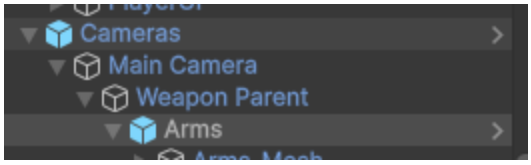
And add your gun prefab next to the other guns shown. Set the scale to however large you want it and zero the pos + rotation. Remember to add the RigLayer and CinemachineImpulse source attributes.

11. Our gun system is based on pickups, so you will have to create a gun pickup prefab and attach the scripts AND colliders (make note of the two separate ones, one with a trigger and one without) shown in the examples here:



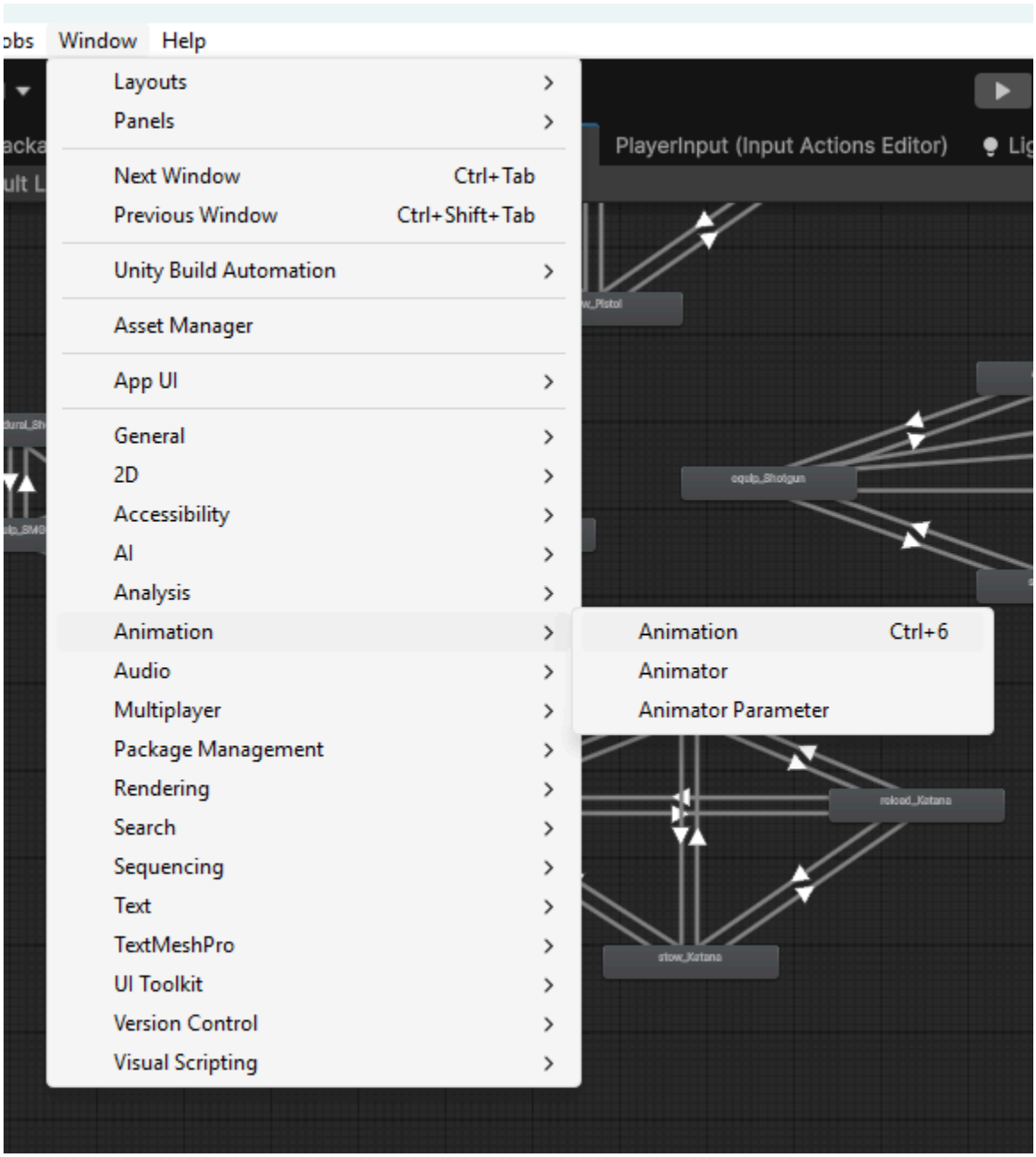
Remember to assign each gun pickup to the right gun Scriptable Object.

12. Go to the arms prefab here:

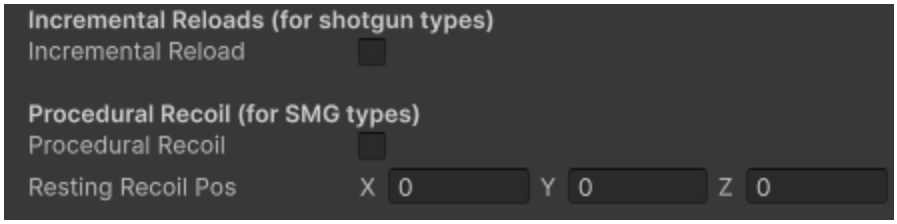


And then in the Ordered Guns list add your gun in the order you set the Scriptable Object IDs. It is important that you match these, otherwise the system won't work.

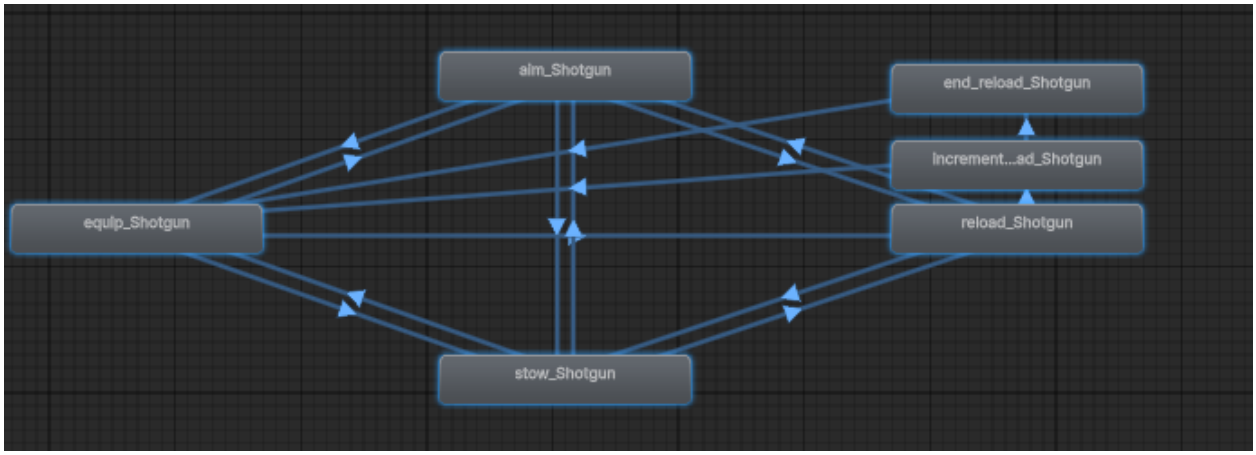
13. Now we need to animate the gun. Select the arms and go to the animator window.



14. Here, you can see all the animation states for all the guns and how the different types of guns result in different structures. If you selected either of these boxes for your gun:



Then copy their respective animation structure. For example if you checked the Incremental Reloads setting, then copy the entire Shotgun animation structure:

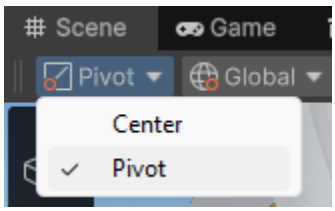


And paste it. Change the block names to the name you specified in GunType. Ex: incremental_reload_Shotgun →incremental_reload“gun type”

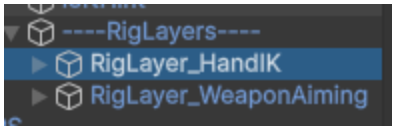
15. For now it’s ok to keep the previous gun’s animations, you can change those after you’ve established your new gun.
16. At this point, colliding with your gun pickup should allow you to carry your new gun, and should have a set of working animations.

Adding New Arms

1. Import new arms to unity, in export settings import only mesh + amature, use FBX unit scale
2. Make sure up top ur setting is PIVOT not CENTER



3. When in scene with arms, attach RigBuilder to your new main arms object
4. Create game object RigLayers underneath main arms, create GameObjects RigLayer_HandIK and RigLayer_WeaponAiming

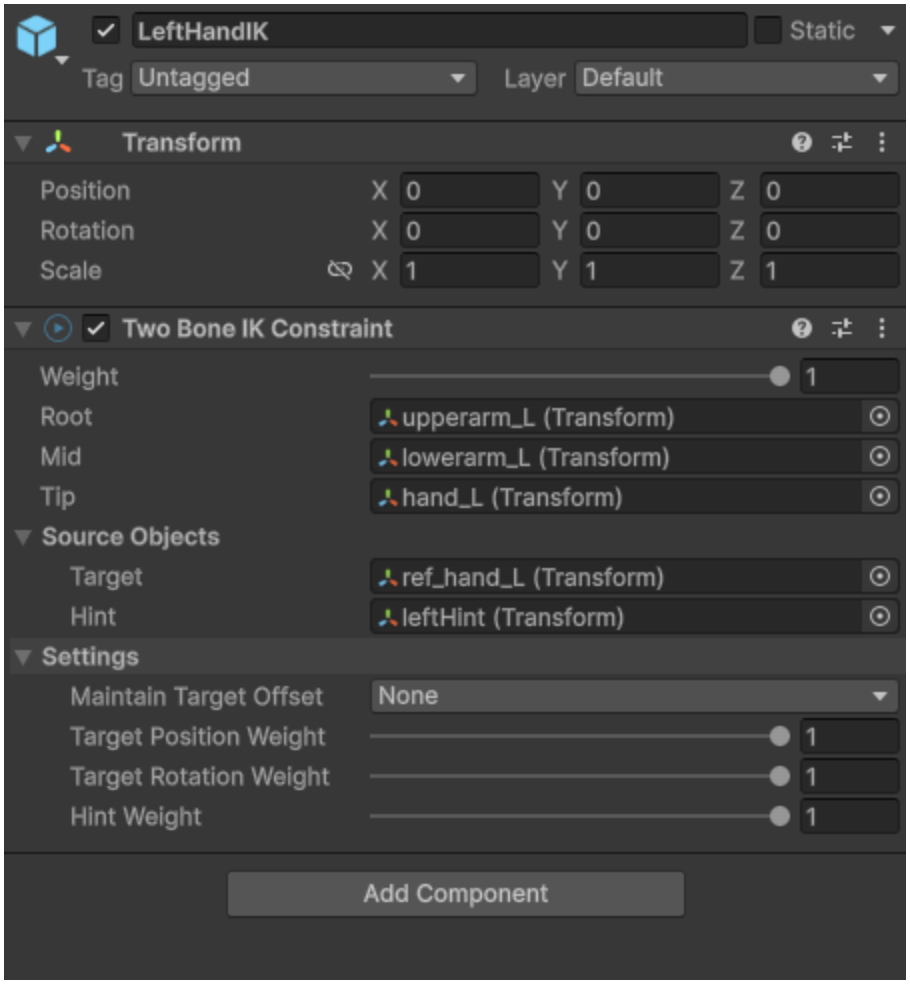


5. Attach the rig component to these
6. Assign these new “rigs” in the main arms RigBuilder section. Put the WeaponAiming Rig before the HandIK



7. Add in any gun you want to start the setup process
8. Create rightHandIK and leftHandIK in RigLayers, attach Two Bone IK Constraint and assign to the physical parts of the arm

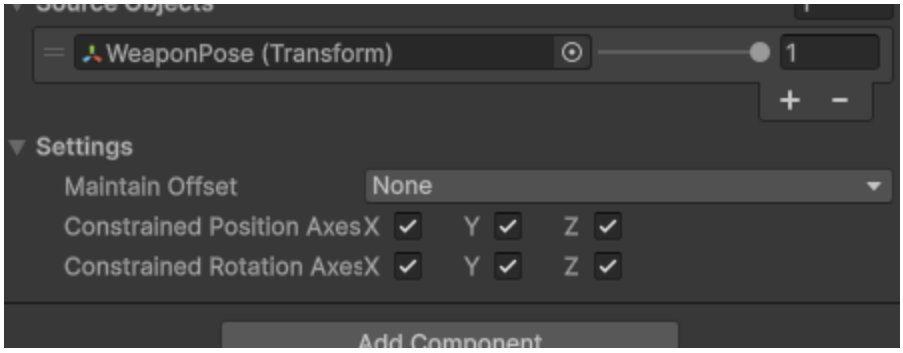




9. Create references ref_hand_L and hand_R underneath your main arms, **must be on the same hierarchy level as the gun you're using**, assign the references to source object in the two bone IK constraints



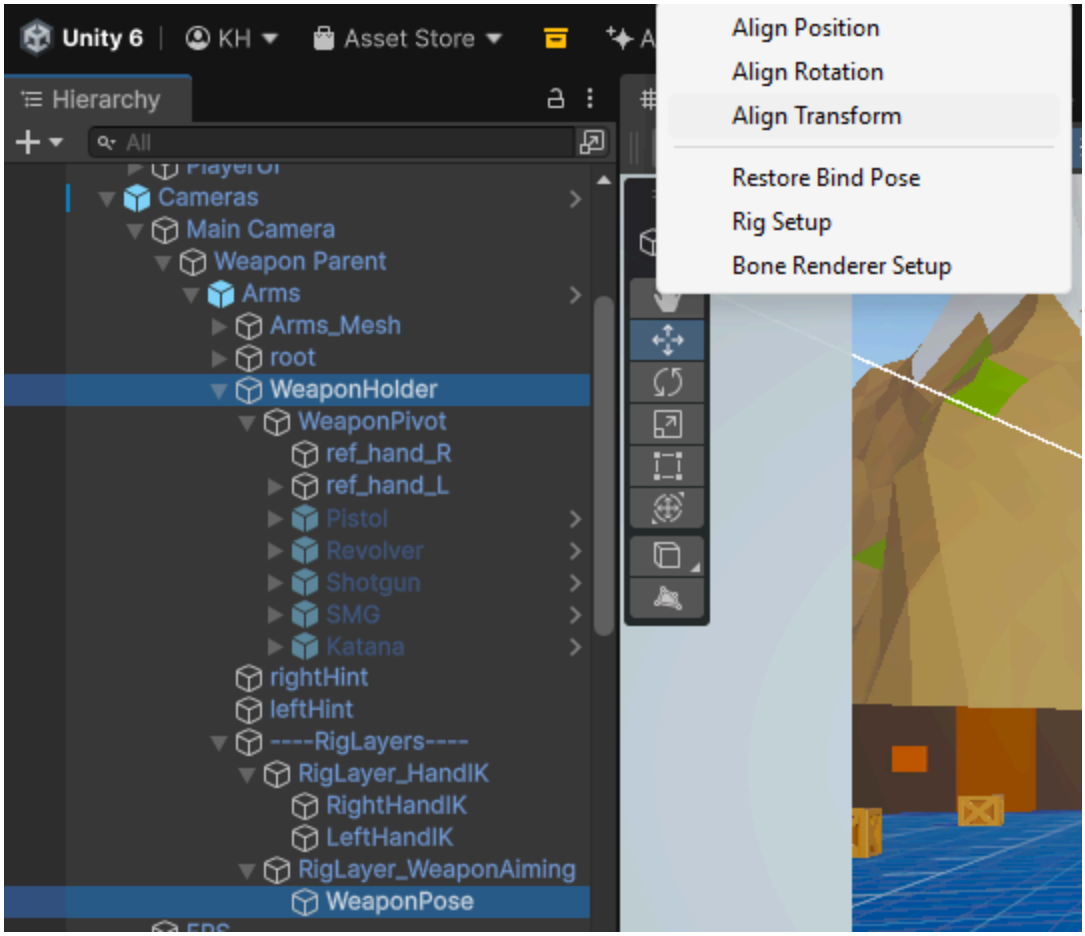
10. Now in scene if you drag the ref_hand_R and ref_hand_L, the physical hands should follow.
11. Create hints under lowerarm, then assign hints in HandIKs to make elbows more natural
12. Create WeaponPivot under gun, and parent it to the gun, make sure its at that corner edge thing of the gun, attach Rig Transform
13. Set WeaponPivot to where you want the gun rest position to be, doesn't really matter because it'll change in animation anyway
14. Create new object WeaponPose for RigLayer_WeaponAiming, multi parent constraint, set constrained object to WeaponPivot. Set source object to WeaponPose



15. Copy the current WeaponPivot transform component values and switch with WeaponPose
16. Create new WeaponHolder parent above WeaponPivot
17. Drag the references and gun into WeaponPivot

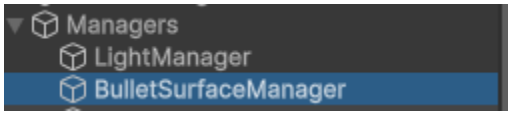


18. Set WeaponHolder to the constrained object instead of WeaponPivot in WeaponPose multi parent constraint
19. Click WeaponHolder and ctrl click WeaponPose -> Animation Rigging, align transform, should see your test gun zip to the the WeaponPivot point you set



20. Set up ref_hands to attach to the gun for testing, copy pos from scene and do it one by one
21. Now in scene if you drag WeaponPivot around you should be able to move both your test gun and your arms
22. Add all the other guns in, set the new GunStateMachine script in Arms, new ProceduralRecoil script in WeaponPivot
23. If arms aren't showing up in scene then go to the materials and turn on

Impact Effects

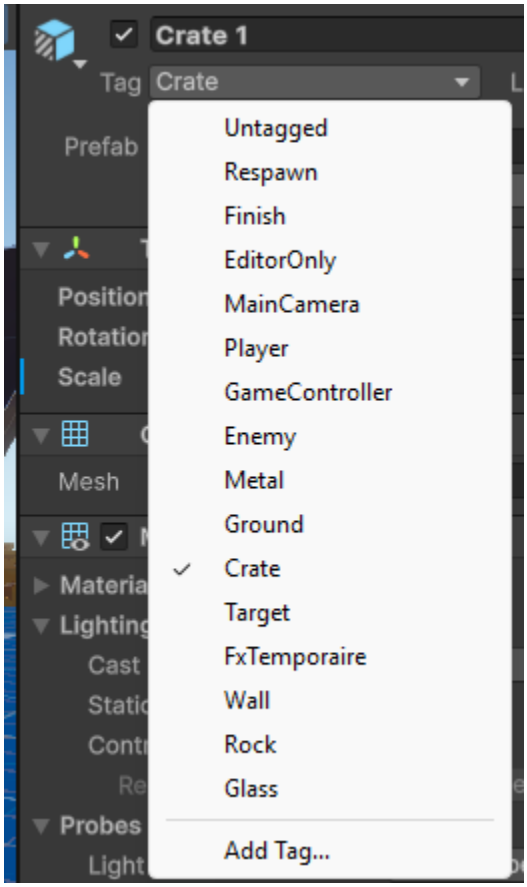


1. This system also uses scriptable objects, where you can assign any GameObject a specific Tag to denote the type of sound and vfx it can create upon impact with a gun.
2. To create a new impact effect, go to this path:



And right-click → create → effects → surface effect. Name it however you'd like, and add it to the list in the BulletSurfaceManager GameObject in scene hierarchy.

3. Now name a surface tag, which you can create a corresponding tag to by clicking on any GameObject and opening the Tag option:



Click Add Tag, and use the same name that you just gave your Surface Effect Scriptable Object

4. Add your desired vfx prefab and list of audio files (which it will pick randomly in an interaction)
5. Apply the tag to the desired objects in scene.

Save and Load

1. F6 to save, F7 to load. If you run a build after running the Playground scene in the Editor, make sure to F6 in that build otherwise it breaks down. And vice versa.